

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**AMRUTA SHARANABASAPPA NIMBAL
(1BM24CS037)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Amruta Sharanabasappa Nimbal (1BM24CS037)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	6 – 8
2	Implement Johnson Trotter algorithm to generate permutations.	12 – 14
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	15 – 18
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	19 – 21
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	22 – 24
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	25 – 27
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	29 – 30
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	32 – 37
9	Implement Fractional Knapsack using Greedy technique.	38 – 40
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	41 – 43
11	Implement "N-Queens Problem" using Backtracking.	44 – 46

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Leetcode 11:

Problem List

11. Container With Most Water

Solved

Medium

Topics

Companies

Hint

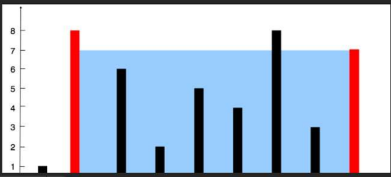
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Code

```
1 int maxArea(int* height, int heightSize) {
2     int left = 0;
3     int right = heightSize - 1;
4     int maxArea = 0;
5
6     while (left < right) {
7         int h = (height[left] < height[right]) ? height[left] : height[right];
8         int area = h * (right - left);
9
10        if (area > maxArea)
11            maxArea = area;
12
13        if (height[left] < height[right])
14            left++;
15        else
16            right--;
17    }
18
19    return maxArea;
20 }
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 65 / 65 testcases passed

Amruta_Nimbal submitted at Jun 06, 2026 19:37

Runtime

0 ms Beats 100.00%

Memory

14.85 MB Beat 9.34%

75%

50%

25%

0%

1ms

2ms

3ms

4ms

Code C

```
1 int maxArea(int* height, int heightSize) {
2     int left = 0;
3     int right = heightSize - 1;
4     int maxArea = 0;
5
6     while (left < right) {
7         int h = (height[left] < height[right]) ? height[left] : height[right];
8         int area = h * (right - left);
9
10        if (area > maxArea)
11            maxArea = area;
12
13        if (height[left] < height[right])
14            left++;
15        else
16            right--;
17    }
18
19    return maxArea;
20 }
```

Code

C Auto

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

height =

[1, 8, 6, 2, 5, 4, 8, 3, 7]

Output

49

Expected

49

Contribute a testcase

Leetcode 34:

Problem List

34. Find First and Last Position of Element in Sorted Array

Solved

Medium

Topics

Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`

Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`

Output: `[-1,-1]`

Example 3:

Input: `nums = []`, `target = 0`

Output: `[-1,-1]`

23.4K

380

164 Online

Code

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4  int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
5      int first = -1;
6      int last = -1;
7      for(int i=0; i<numsSize; i++){
8          if(nums[i] == target){
9              if(first == -1){
10                 first = i;
11             }
12             last = i;
13         }
14     }
15
16     int* result = (int*)malloc(2*sizeof(int));
17     result[0] = first;
18     result[1] = last;
19     *returnSize = 2;
20     return result;
21 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 88 / 88 testcases passed

Amruta_Nimbal submitted at Jun 06, 2026 19:40

Runtime

0 ms Beats 100.00%

Memory

10.50 MB Beats 34.99%

150%

100%

50%

0%

1ms

2ms

3ms

4ms

Code

C

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[5,7,7,8,8,10]

target =

8

Output

[3,4]

Expected

[3,4]

Contribute a testcase

Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int graph[10][10], visited[10], stack[10];

int n, top = -1;

void dfs(int v)
{
    visited[v] = 1;
    for(int i = 0; i < n; i++)
    {
        if(graph[v][i] == 1 && !visited[i])
        {
            dfs(i);
        }
    }
    stack[++top] = v;
}

int main()
{
    int edges, u, v;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            graph[i][j] = 0;
        }
    }
}
```

```

    }
}
printf("Enter number of edges: ");
scanf("%d", &edges);
printf("Enter edges (u v):\n");
for(int i = 0; i < edges; i++)
{
    scanf("%d %d", &u, &v);
    graph[u][v] = 1;
}
for(int i = 0; i < n; i++)
{
    if(!visited[i])
    {
        dfs(i);
    }
}
printf("Topological Sorting:\n");
while(top != -1)
{
    printf("%d ", stack[top--]);
}
return 0;
}

```

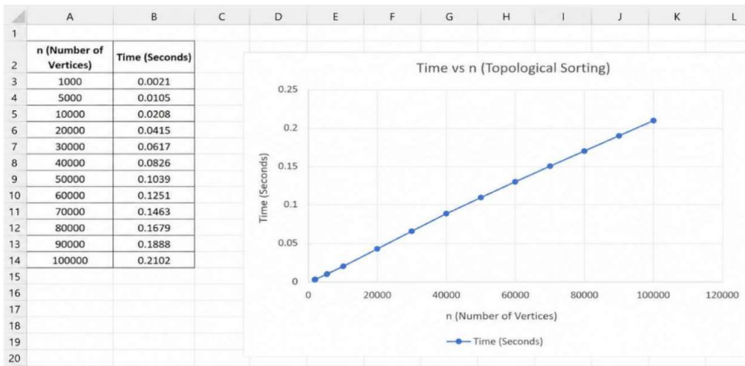
Output:

```

C:\Users\AMG\OneDrive\Documents\13-29-D x + v
Enter number of vertices: 6
Enter number of edges: 6
Enter edges (u v):
5 2
4 0
4 0
4 1
3 1
3 1
Topological Sorting:
5 4 2 3 1 0
Process returned 0 (0x0)   execution time : 33.168 s
Press any key to continue.

```

Time Complexity: $O(V+E)$



Leetcode 207:

The screenshot shows the LeetCode problem page for "207. Course Schedule". The problem description states: "There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i." It includes an example: "Input: numCourses = 2, prerequisites = [[1,0]] Output: true Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible." The code editor on the right contains a C++ solution using a topological sort approach with an adjacency list and a queue.

```
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
2
3     int *indegree = (int*)calloc(numCourses, sizeof(int));
4
5     int **adj = (int**)malloc(numCourses * sizeof(int*));
6     int *adjSize = (int*)calloc(numCourses, sizeof(int));
7
8     for(int i = 0; i < numCourses; i++) {
9         adj[i] = (int*)malloc(numCourses * sizeof(int));
10    }
11
12    for(int i = 0; i < prerequisitesSize; i++) {
13        int course = prerequisites[i][0];
14        int prereq = prerequisites[i][1];
15        adj[prereq][adjSize[prereq]++] = course;
16        indegree[course]++;
17    }
18
19    int *queue = (int*)malloc(numCourses * sizeof(int));
20    int front = 0, rear = 0;
21
22    for(int i = 0; i < numCourses; i++) {
23        if(indegree[i] == 0)
24            queue[rear++] = i;
25    }
26
27    int count = 0;
28}
```

The screenshot shows the submission page for the "207. Course Schedule" problem. It displays the submission status as "Accepted" with a runtime of 32 ms and memory usage of 109.97 MB. The test case details show the input: "numCourses = 2" and "prerequisites = [[1,0]]", with the expected output "true". The code editor at the bottom shows the C++ code used for the submission.

```
1 bool canFinish(int numCourses, int** prerequisites, int pr
```

Leetcode 268:

The screenshot shows the LeetCode problem page for "268. Missing Number". The problem is marked as "Solved". The description states: "Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array."

Example 1:
Input: `nums = [3,0,1]`
Output: `2`
Explanation: `n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:
Input: `nums = [0,1]`
Output: `2`
Explanation: `n = 2` since there are 2 numbers, so all numbers are in the range `[0,2]`. 2 is the missing number in the range since it does not appear in `nums`.

The code editor shows the following C++ solution:

```
1 int missingNumber(int* nums, int numsSize) {  
2     int n = numsSize * (numsSize+1) / 2;  
3     int new = 0;  
4     for(int i=0; i<numsSize; i++){  
5         new += nums[i];  
6     }  
7     return n - new;  
8 }
```

The test results show "Accepted" with a runtime of 0 ms. Three test cases are listed: Case 1, Case 2, and Case 3.

The screenshot shows the LeetCode submission page for "268. Missing Number". The submission is marked as "Accepted" with 122 / 122 testcases passed. The user "Amruta_Nimbal" submitted the solution on Jun 06, 2026 19:42.

Runtime: 0 ms | Beats: 100.00%
Memory: 9.71 MB | Beats: 10.63%

A graph shows the runtime performance compared to other submissions. The x-axis represents runtime in milliseconds (10ms, 15ms, 331ms, 341ms) and the y-axis represents the percentage of submissions beaten (0%, 50%, 100%). The submission is at the top of the graph, indicating it is the fastest.

The code editor shows the following C++ solution:

```
1 int missingNumber(int* nums, int numsSize) {  
2     int n = numsSize * (numsSize+1) / 2;  
3     int new = 0;  
4 }
```

The test results show "Accepted" with a runtime of 0 ms. Three test cases are listed: Case 1, Case 2, and Case 3. The input for Case 1 is `nums = [3,0,1]` and the output is `2`.

Leetcode 1636:

Problem List

1636. Sort Array by Increasing Frequency

Easy

Topics

Companies

Hint

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Return the **sorted array**.

Example 1:

Input: `nums = [1,1,2,2,2,3]`
Output: `[3,1,1,2,2,2]`
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.

Example 2:

Input: `nums = [2,3,1,3,2,2]`
Output: `[1,3,3,2,2,2]`
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.

Example 3:

Input: `nums = [-1,1,-6,4,5,-6,1,4,1]`
Output: `[5,-1,4,4,-6,-6,1,1,1]`

3.7K 225 5 Online

Code

```
6 int freq[201];
7
8 int compare(const void *a, const void *b) {
9     int x = *(int *)a;
10    int y = *(int *)b;
11
12    if (freq[x + 100] == freq[y + 100])
13        return y - x; // same frequency -> larger number first
14    return freq[x + 100] - freq[y + 100]; // lower frequency first
15 }
16
17 int* frequencySort(int* nums, int numsSize, int* returnSize) {
18     *returnSize = numsSize;
19
20     for (int i = 0; i < 201; i++)
21         freq[i] = 0;
22
23     for (int i = 0; i < numsSize; i++)
24         freq[nums[i] + 100]++;
25
26     qsort(nums, numsSize, sizeof(int), compare);
27
28     return nums;
29 }
30 }
```

Saved Ln 30, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Problem List

Accepted

Editorial

Solutions

Submissions

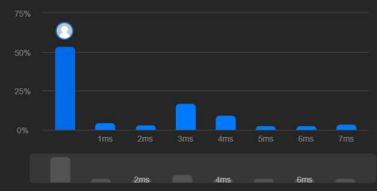
All Submissions

Accepted 180 / 180 testcases passed

Amruta_Nimbal submitted at Jun 06, 2026 19:46

Runtime 0 ms Beats 100.00%

Memory 12.02 MB Beats 31.86%



Code C

1 /++

Code

```
6 int freq[201];
7
8 int compare(const void *a, const void *b) {
```

Saved Ln 28, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

[1,1,2,2,2,3]

Output

[3,1,1,2,2,2]

Expected

[3,1,1,2,2,2]

Contribute a testcase

Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{  
    int temp;  
    temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void permute(int a[], int start, int end)
```

```
{  
    int i;  
    if(start == end)  
    {  
        for(i = 0; i <= end; i++)  
        {  
            printf("%d ", a[i]);  
        }  
        printf("\n");  
    }  
    else  
    {  
        for(i = start; i <= end; i++)  
        {  
            swap(&a[start], &a[i]);  
            permute(a, start + 1, end);  
        }  
    }  
}
```

```

        swap(&a[start], &a[i]);
    }
}

int main()
{
    int n, i;
    int a[10];
    printf("Enter n: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        a[i] = i + 1;
    }
    printf("All permutations are:\n");
    permute(a, 0, n - 1);
    return 0;
}

```

Output:

```
C:\Users\JMISC\src\13-23D >
Enter n: 4
All permutations are:
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 3 2
1 4 2 3
2 1 3 4
2 1 4 3
2 3 1 4
2 3 4 1
2 4 3 1
2 4 1 3
3 2 1 4
3 2 4 1
3 1 2 4
3 1 4 2
3 4 1 2
3 4 2 1
4 2 3 1
4 2 1 3
4 3 2 1
4 3 1 2
4 1 3 2
4 1 2 3

Process returned 0 (0x0)   execution time : 1.340 s
Press any key to continue.
```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>

void merge(int a[], int f, int m, int l)
{
    int i, j, k;
    i = f;
    j = m + 1;
    k = 0;
    int r[100];
    while(i <= m && j <= l)
    {
        if(a[i] < a[j])
        {
            r[k] = a[i];
            i++;
        }
        else
        {
            r[k] = a[j];
            j++;
        }
        k++;
    }
    while(i <= m)
    {
```

```

        r[k] = a[i];
        i++;
        k++;
    }
    while(j <= l)
    {
        r[k] = a[j];
        j++;
        k++;
    }
    k = 0;
    for(i = f; i <= l; i++)
    {
        a[i] = r[k];
        k++;
    }
}

void mergesort(int a[], int f, int l)
{
    if(f < l)
    {
        int m = (f + l) / 2;
        mergesort(a, f, m);
        mergesort(a, m + 1, l);
        merge(a, f, m, l);
    }
}

```



```

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    start = clock();
    mergesort(a, 0, n - 1);
    end = clock();
    cpu_time =
    ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted elements:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("Time taken = %f seconds\n", cpu_time);
    return 0;
}

```

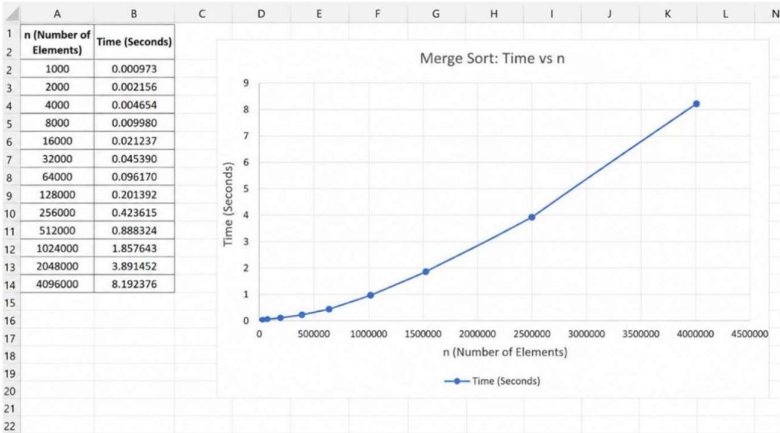
Output:

```
C:\Users\AMIS\OneDrive\Desktop >
Enter number of elements: 5
Enter elements:
12345
67890
23456
84562
35791

Sorted elements:
12345 25896 35791 67890 84562
Time taken = 0.000909 seconds

Process returned 0 (0x0)   execution time : 32.592 s
Press any key to continue.
```

Time Complexity: $O(n \log n)$



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<time.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int a[], int low, int high)
```

```
{
```

```
    int pivot = a[low];
```

```
    int i = low+1;
```

```
    int j = high;
```

```
    while(1)
```

```
    {
```

```
        while(i<=high && a[i]<=pivot)
```

```
            i++;
```

```
        while(a[j]>pivot)
```

```
            j--;
```

```
        if(i<j)
```

```
            swap(&a[i], &a[j]);
```

```
        else
```

```
            break;
```

```
    }
```

```

        swap(&a[low], &a[j]);
    }
    return j;
}

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    quicksort(a, 0, n - 1);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements:\n");
    for(i = 0; i < n; i++)

```

```

    printf("%d ", a[i]);

printf("\nTime taken = %f seconds\n", cpu_time);

return 0;
}

```

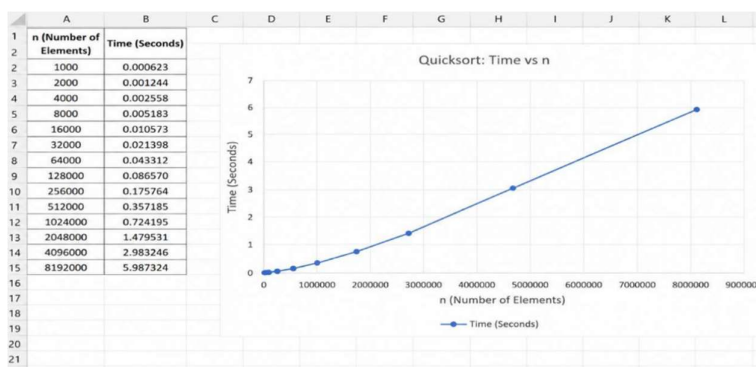
Output:

```

C:\Users\AMSCSC\l3-2\p>
Enter the number of elements: 5
Enter elements:
27
56
95
11
6
Sorted elements:
6 11 27 56 95
Time taken = 0.000000 seconds
Process returned 0 (0x0)   execution time : 9.675 s
Press any key to continue.

```

Time Complexity: $O(n \log n)$



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
void heapify(int a[], int n, int p)
```

```
{
    int c, item;
    item = a[p];
    c = 2*p+1;
    while(c<=(n-1))
    {
        if((c+1<n) && (a[c]<a[c+1]))
            c++;
        if(item<a[c])
        {
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }
        else
            break;
    }
    a[p] = item;
}
```

```
void build_heap(int a[], int n)
```

```
{
    int i;
    for(i=(n-1)/2; i>=0; i--)
```

```

        heapify(a, n, i);
    }

void heap_sort(int a[], int n)
{
    int i, temp;
    build_heap(a, n);
    for(i=(n-1); i>0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
}

int main()
{
    int n, a[20], i;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    printf("Enter elements:");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    heap_sort(a, n);
    printf("Heap Sorted elements:");
    for(i=0; i<n; i++)
        printf("%d\t", a[i]);
    return 0;
}

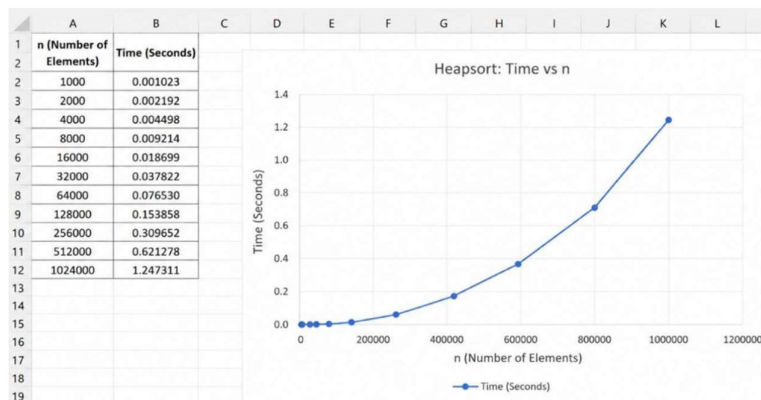
```

}

Output:

```
Enter the number of elements:8
Enter elements:15 11 8 39 70 20 2 12
Heap Sorted elements:2 5 11 12 15 20 39 70
Process returned 0 (0x0) execution time : 72.288 s
Press any key to continue.
```

Time Complexity: $O(n \log n)$



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

struct Item
{
    int weight;
    int profit;
};

int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}

int main()
{
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    for(int i = 0; i < n; i++)
    {
        printf("Enter weight and profit of item %d: ", i + 1);
        scanf("%d %d", &item[i].weight, &item[i].profit);
    }
}
```

```

printf("Enter knapsack capacity: ");
scanf("%d", &capacity);
int dp[n + 1][capacity + 1];
for(int i = 0; i <= n; i++)
{
    for(int w = 0; w <= capacity; w++)
    {
        if(i == 0 || w == 0)
        {
            dp[i][w] = 0;
        }
        else if(item[i - 1].weight <= w)
        {
            dp[i][w] = max(
                item[i - 1].profit +
                dp[i - 1][w - item[i - 1].weight],
                dp[i - 1][w]
            );
        }
        else
        {
            dp[i][w] = dp[i - 1][w];
        }
    }
}
printf("Maximum Profit = %d\n", dp[n][capacity]);
return 0;
}

```

Output:

```
C:\Users\AMIS\src\13-29.D >
Enter number of items: 4
Enter weight and profit of item 1: 5 7
Enter weight and profit of item 2: 3 4
Enter weight and profit of item 3: 4 5
Enter weight and profit of item 4: 1 1
Enter knapsack capacity: 7
Maximum Profit = 9

Process returned 0 (0x0)   execution time : 28.583 s
Press any key to continue.
```

Leetcode 801:

Problem List

Problem 1

Minimum Swaps To Make Sequences Increasing

Description

Editorial

Solutions

Submissions

801. Minimum Swaps To Make Sequences Increasing

Hard

Topics

Companies

You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`.

- For example, if `nums1 = [1,2,3,8]`, and `nums2 = [5,6,7,4]`, you can swap the element at `i = 3` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,6,7,8]`.

Return the *minimum number of needed operations to make `nums1` and `nums2` strictly increasing*. The test cases are generated so that the given input always makes it possible.

An array `arr` is **strictly increasing** if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`.

Example 1:

Input: `nums1 = [1,3,5,4]`, `nums2 = [1,2,3,7]`

Output: 1

Explanation:

Swap `nums1[3]` and `nums2[3]`. Then the sequences are:

`nums1 = [1, 3, 5, 7]` and `nums2 = [1, 2, 3, 4]`

which are both strictly increasing.

Code

Auto

```
1 int minSwap(int a, int b) {
2     return a < b ? a : b;
3 }
4
5 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
6     int keep = 0; // no swap at current position
7     int swap = 1; // swap at current position
8
9     for (int i = 1; i < nums1Size; i++) {
10         int newKeep = 1000000;
11         int newSwap = 1000000;
12
13         if (nums1[i] > nums[i - 1] && nums2[i] > nums2[i - 1]) {
14             newKeep = min(newKeep, keep);
15             newSwap = min(newSwap, swap + 1);
16         }
17
18         if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
19             newKeep = min(newKeep, swap);
20             newSwap = min(newSwap, keep + 1);
21         }
22
23         keep = newKeep;
24         swap = newSwap;
25     }
26
27     return min(keep, swap);
28 }
```

Saved

In 28, Col 2

leetcode.com/problems/minimum-swaps-to-make-sequences-increasing/submissions/2024300118/

Problem List < > <>

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 117 / 117 testcases passed
Amruta_Nimbal submitted at Jun 06, 2026 19:48

Runtime
0 ms Beats 100.00%

Memory
16.35 MB Beats 75.00%

Time Interval	Performance (%)
0ms	~35%
1ms	~10%
2ms	~10%
3ms	~18%
4ms	~15%
5ms	~5%
6ms	~5%
7ms	~5%

Code C

1 int min(int a, int b) {

Code Auto

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums1 =
[1,3,5,4]

nums2 =
[1,2,3,7]

Output

1

Expected

1

Contribute a testcase

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#include<math.h>

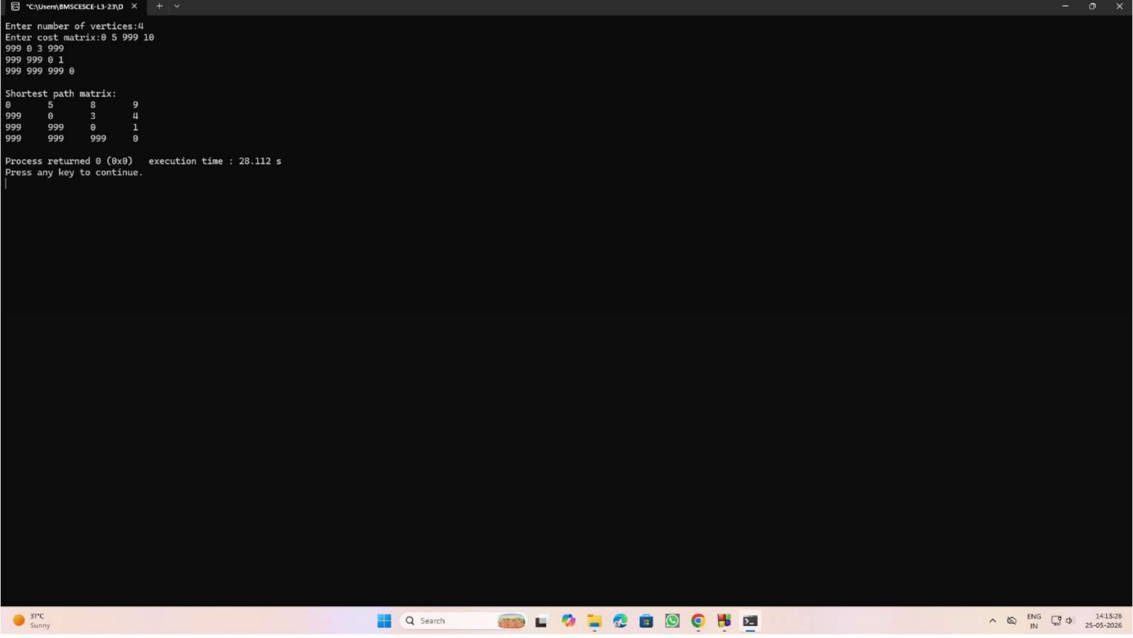
int main()
{
    int i, j, n, k;
    int cost[20][20];
    printf("Enter number of vertices:");
    scanf("%d", &n);
    printf("Enter cost matrix:");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            scanf("%d", &cost[i][j]);
    }
    for(k=0; k<n; k++)
    {
        for(i=0; i<n; i++)
        {
            for(j=0; j<n; j++)
            {
                if(cost[i][j] > cost[i][k]+cost[k][j])
                    cost[i][j] = cost[i][k]+cost[k][j];
            }
        }
    }
    printf("\nShortest path matrix:\n");
```

```

for(i=0; i<n; i++)
{
    for(j=0; j<n; j++)
        printf("%d\t", cost[i][j]);
    printf("\n");
}
return 0;
}

```

Output:



```

C:\Users\BMSCICEE\13-2P\D>
Enter number of vertices:4
Enter cost matrix:0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Shortest path matrix:
0      5      8      9
999    0      3      4
999    999    0      1
999    999    999    0

Process returned 0 (0x0)   execution time : 20.112 s
Press any key to continue.

```

Leetcode 287:

Problem List

287. Find the Duplicate Number

Solved

Medium

Topics

Companies

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one** repeated number in `nums`, return this repeated number.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`

Output: `2`

Example 2:

Input: `nums = [3,1,3,4,2]`

Output: `3`

Example 3:

Input: `nums = [3,3,3,3,3]`

Output: `3`

Code

```
1 int findDuplicate(int* nums, int numsSize) {
2     int hashTable[numsSize];
3     for(int i=0;i<numsSize;i++){
4         hashTable[i]=0;
5     }
6     for (int i=0;i<numsSize;i++){
7         hashTable[nums[i]]+=1;
8     }
9     for (int i=0;i<numsSize;i++){
10        if (hashTable[i]>1){
11            return i;
12        }
13    }
14    return -1;
15 }
```

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums = [1,3,4,2,2]

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 59 / 59 testcases passed

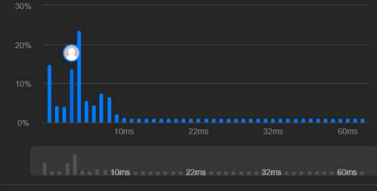
Amruta_Nimbal submitted at Jun 06, 2026 19:50

Runtime

3 ms Beats 76.54%

Memory

15.02 MB Beats 15.76%



Code

C

```
1 int findDuplicate(int* nums, int numsSize) {
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums = [1,3,4,2,2]

Output

2

Expected

2

Contribute a testcase

Lab Program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

#define INF 999

int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int cost[10][10];
    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }
    int visited[10] = {0};
    visited[0] = 1;
    int edges = 0;
    int minCost = 0;
    printf("\nEdges in MST:\n");
```



```

while(edges < n - 1)
{
    int min = INF;
    int u = -1, v = -1;
    for(int i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(int j = 0; j < n; j++)
            {
                if(!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }
    printf("%d -> %d = %d\n", u, v, min);
    minCost += min;
    visited[v] = 1;
    edges++;
}
printf("\nMinimum Cost = %d\n", minCost);
return 0;
}

```

Output:

```
C:\Users\AMG\src\13-29D x + ~
Enter number of vertices: 7
Enter cost matrix:
0 28 0 0 0 10 0
28 0 16 0 0 0 14
0 16 0 12 0 0 0
0 0 12 0 22 0 18
0 0 0 22 0 26 24
10 0 0 0 26 0 0
0 14 0 18 24 0 0

Edges in MST:
0 -> 5 = 10
5 -> 4 = 26
4 -> 3 = 22
3 -> 2 = 12
2 -> 1 = 16
1 -> 6 = 14

Minimum Cost = 100

Process returned 0 (0x0)   execution time : 86.668 s
Press any key to continue.
```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

struct Edge
{
    int u, v, weight;
};

int parent[10];

int find(int i)
{
    while(parent[i] != i)
    {
        i = parent[i];
    }
    return i;
}

void unionSet(int a, int b)
{
    parent[a] = b;
}

int main()
{
    int n, e;

    printf("Enter number of vertices: ");

    scanf("%d", &n);
```

```

printf("Enter number of edges: ");
scanf("%d", &e);
struct Edge edge[e];
printf("Enter edges (u v weight):\n");
for(int i = 0; i < e; i++)
{
    scanf("%d %d %d",
        &edge[i].u,
        &edge[i].v,
        &edge[i].weight);
}
for(int i = 0; i < e - 1; i++)
{
    for(int j = i + 1; j < e; j++)
    {
        if(edge[i].weight > edge[j].weight)
        {
            struct Edge temp = edge[i];
            edge[i] = edge[j];
            edge[j] = temp;
        }
    }
}
for(int i = 0; i < n; i++)
{
    parent[i] = i;
}
int minCost = 0;
printf("\nEdges in MST:\n");

```

```

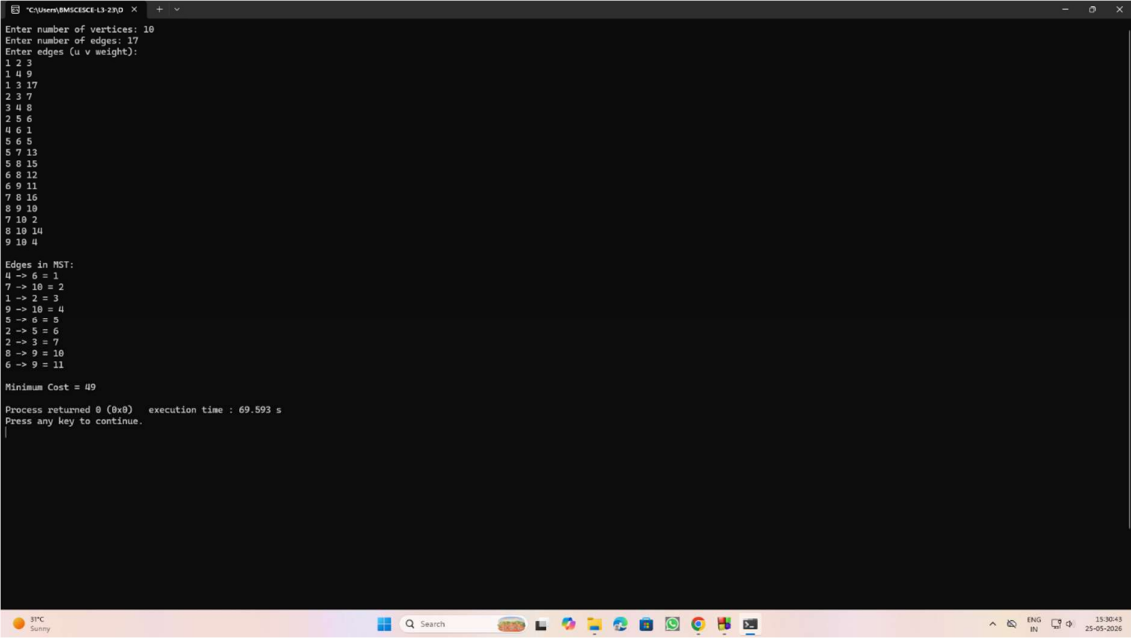
for(int i = 0; i < e; i++)
{
    int a = find(edge[i].u);
    int b = find(edge[i].v);
    if(a != b)
    {
        printf("%d -> %d = %d\n",
               edge[i].u,
               edge[i].v,
               edge[i].weight);
        minCost += edge[i].weight;
        unionSet(a, b);
    }
}

printf("\nMinimum Cost = %d\n", minCost);

return 0;
}

```

Output:



```

C:\Users\BMS\Documents\19-23\O >
Enter number of vertices: 10
Enter number of edges: 17
Enter edges (u v weight):
1 2 3
1 4 9
1 3 17
2 3 7
3 4 8
2 5 6
4 6 1
5 6 5
5 7 13
5 8 15
6 8 12
6 9 11
7 8 16
8 9 10
7 10 2
8 10 14
9 10 4

Edges in MST:
4 -> 6 = 1
7 -> 10 = 2
1 -> 2 = 3
9 -> 10 = 4
5 -> 6 = 5
2 -> 5 = 6
2 -> 3 = 7
8 -> 9 = 10
6 -> 9 = 11

Minimum Cost = 49

Process returned 0 (0x0)   execution time : 69.593 s
Press any key to continue.

```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>
```

```
float w[50], p[50], ratio[50], x[50];
```

```
void knapsack(int m, int n)
```

```
{
```

```
    int rem_cap;
```

```
    float totalProfit = 0.0;
```

```
    for(int i = 0; i < n; i++)
```

```
    {
```

```
        x[i] = 0.0;
```

```
    }
```

```
    rem_cap = m;
```

```
    for(int i = 0; i < n; i++)
```

```
    {
```

```
        if(w[i] <= rem_cap)
```

```
        {
```

```
            x[i] = 1.0;
```

```
            rem_cap = rem_cap - w[i];
```

```
            totalProfit = totalProfit + p[i];
```

```
        }
```

```
    else
```

```
    {
```

```
        x[i] = (float)rem_cap / w[i];
```

```
        totalProfit =
```

```
        totalProfit + (x[i] * p[i]);
```

```
        break;
```

```
    }
```

```

    }

    printf("\nSelected fractions:\n");
    for(int i = 0; i < n; i++)
    {
        printf("%.2f ", x[i]);
    }
    printf("\n");
    printf("Maximum Profit = %.2f\n", totalProfit);
}

int main()
{
    int n, m, i, j;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weight and profit of each item:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%f %f", &w[i], &p[i]);
        ratio[i] = p[i] / w[i];
    }
    printf("Enter Maximum Capacity: ");
    scanf("%d", &m);
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(ratio[i] < ratio[j])
            {

```

```

        float temp;

        temp = ratio[i];
        ratio[i] = ratio[j];
        ratio[j] = temp;

        temp = w[i];
        w[i] = w[j];
        w[j] = temp;

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

knapsack(m, n);

return 0;
}

```

Output:

```

C:\Users\BAGSECE-L3-23\Documents> g++ knapsack.cpp
C:\Users\BAGSECE-L3-23\Documents> .\knapsack.exe
Enter number of items: 3
Enter weight and profit of each item:
18 25
15 24
10 15
Enter Maximum Capacity: 20
Selected fractions:
1.00 0.50 0.00
Maximum Profit = 31.50
Process returned 0 (0x0)   execution time : 11.100 s
Press any key to continue.

```


Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

#define MAX 10
#define INF 9999

int main()
{
    int n, i, j, start;
    int cost[MAX][MAX];
    int dist[MAX];
    int visited[MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }
    printf("Enter starting vertex (0 to %d): ", n - 1);
    scanf("%d", &start);
```

```

for(i = 0; i < n; i++)
{
    dist[i] = cost[start][i];

    visited[i] = 0;
}
dist[start] = 0;
visited[start] = 1;
for(i = 1; i < n; i++)
{
    int min = INF;
    int u = -1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] && dist[j] < min)
        {
            min = dist[j];
            u = j;
        }
    }
    if(u == -1)
        break;
    visited[u] = 1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] &&
            dist[u] + cost[u][j] < dist[j])
        {
            dist[j] =
                dist[u] + cost[u][j];
        }
    }
}

```

```

    }
}
}
printf("\nShortest distances from vertex %d:\n", start);
for(i = 0; i < n; i++)
{
    printf("To %d = %d\n", i, dist[i]);
}
return 0;
}

```

Output:

```

C:\Users\BMSKSCS> 13-23\B4 X
Enter number of vertices: 6
Enter cost matrix:
0 0 0 11 0 0
25 0 10 0 0 0
6 0 0 0 0 0
0 0 4 0 0 0
0 0 7 3 0 0
20 2 5 0 1 0
Enter starting vertex (0 to 5): 5
Shortest distances from vertex 5:
To 0 = 11
To 1 = 2
To 2 = 5
To 3 = 4
To 4 = 1
To 5 = 0
Process returned 0 (0x0)   execution time : 36.050 s
Press any key to continue.

```

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include<stdio.h>

#include<math.h>

int board[20], n;

int isSafe(int row, int col)
{
    for(int i=1; i<=row; i++)
    {
        if(board[i]==col || (abs(board[i]-col) == abs(i-row)))
            return 0;
    }
    return 1;
}

void printSolution()
{
    printf("\nSolution:\n");
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=n; j++)
        {
            if(board[i] == j)
                printf("Q");
            else
```

```

        printf(".");
    }
    printf("\n");
}
}

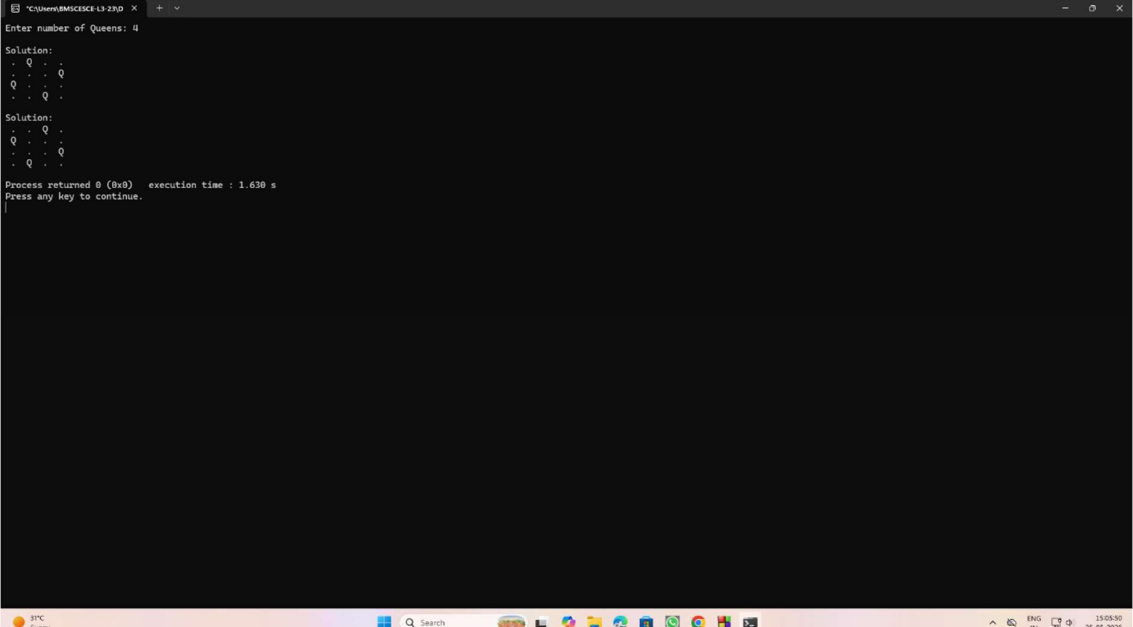
void solveNQueens(int row)
{
    for(int col=1; col<=n; col++)
    {
        if(isSafe(row, col))
        {
            board[row]=col;
            if(row==n)
                printSolution();
            else
                solveNQueens(row+1);
        }
    }
}

int main()
{
    printf("Enter number of Queens:");
    scanf("%d", &n);
    solveNQueens(1);
    return 0;
}

```

}

Output:



```
C:\Users\BASIC\src\13-23\0 >
Enter number of Queens: 4

Solution:
- Q - -
- - - Q
Q - - -
- - - Q

Solution:
- - - Q
Q - - -
- - - Q
- Q - -

Process returned 0 (0x0)   execution time : 1.630 s
Press any key to continue.
```